

Báo cáo bài tập ngày 26/05/2026

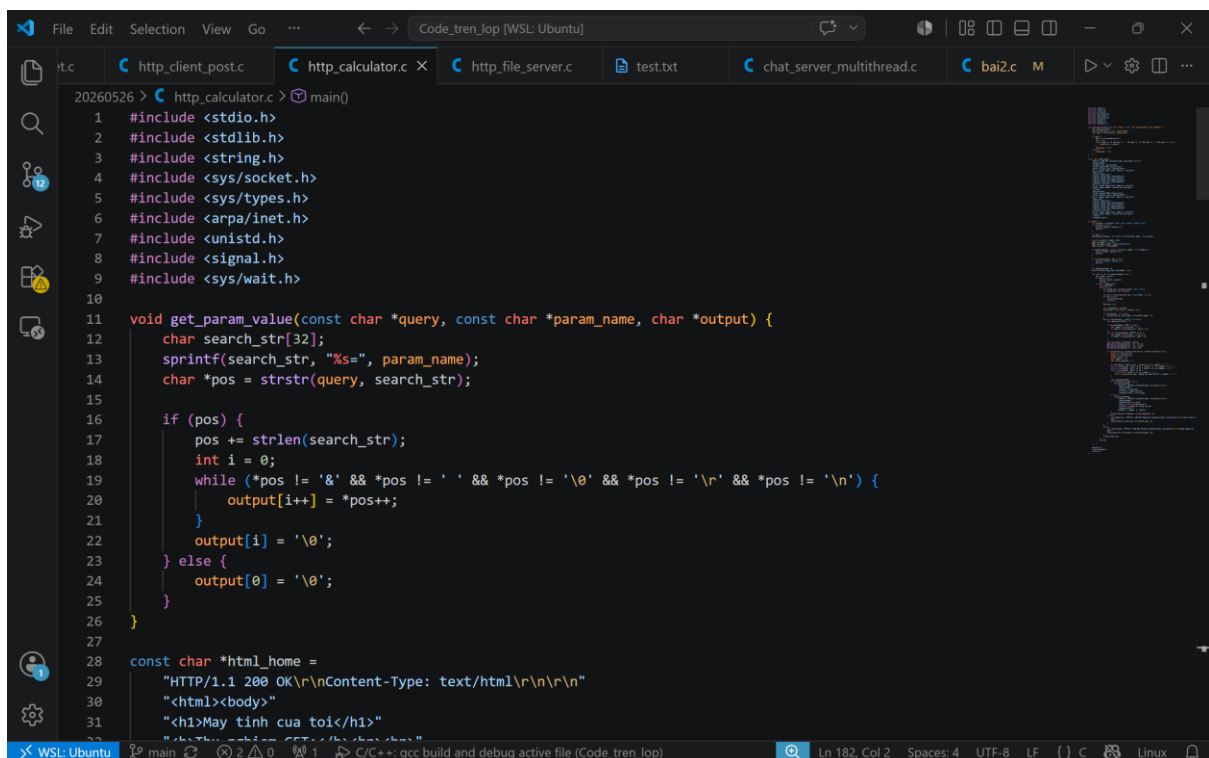
Họ và tên: Nguyễn Kim Tùng Quân

MSSV: 20235407

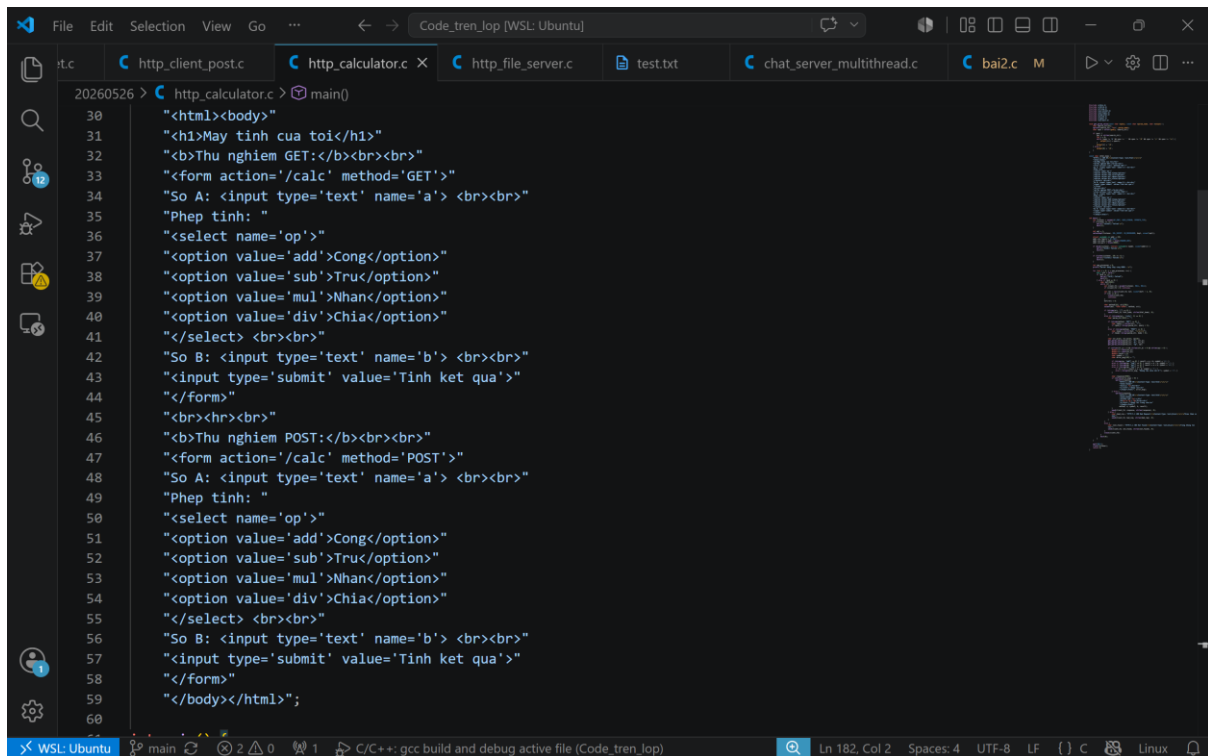
Bài 1: Tạo trang web thực hiện các phép tính cộng, trừ, nhân, chia và trả lại kết quả cho trình duyệt:

- + Nhận tham số từ lệnh GET
- + Nhận tham số từ lệnh POST
- + Các tham số bao gồm toán tử (cộng, trừ, nhân, chia) và 2 toán hạng
- + Server trả lại kết quả là trang web hiển thị phép tính và kết quả.

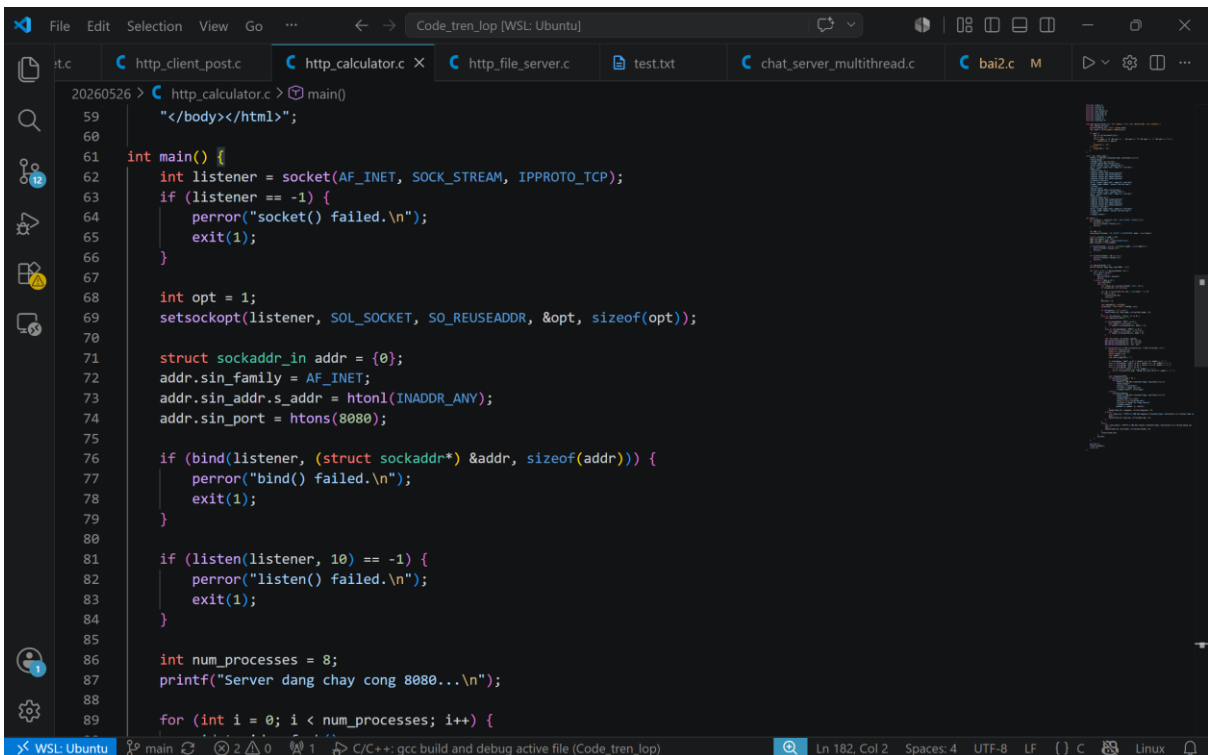
Code:



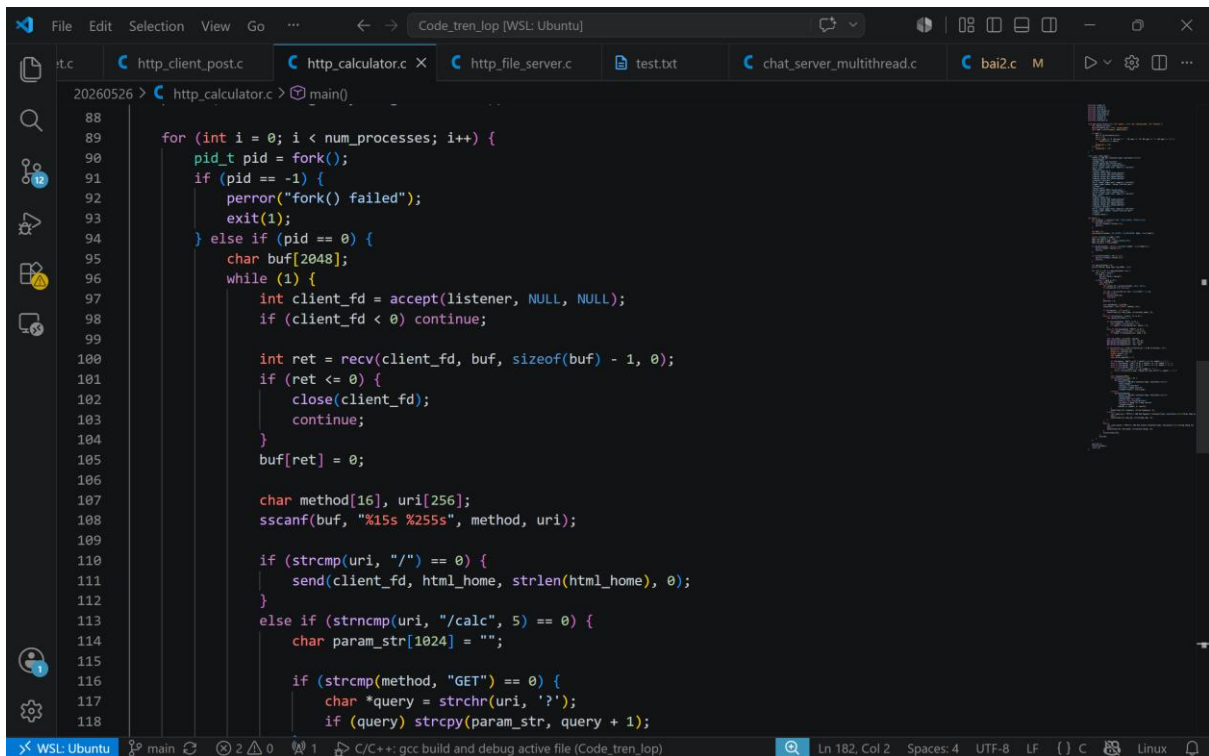
```
20260526 > C http_calculator.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/socket.h>
5  #include <sys/types.h>
6  #include <arpa/inet.h>
7  #include <unistd.h>
8  #include <signal.h>
9  #include <sys/wait.h>
10
11 void get_param_value(const char *query, const char *param_name, char *output) {
12     char search_str[32];
13     sprintf(search_str, "%s=", param_name);
14     char *pos = strstr(query, search_str);
15
16     if (pos) {
17         pos += strlen(search_str);
18         int i = 0;
19         while (*pos != '=' && *pos != ' ' && *pos != '\0' && *pos != '\r' && *pos != '\n') {
20             output[i++] = *pos++;
21         }
22         output[i] = '\0';
23     } else {
24         output[0] = '\0';
25     }
26 }
27
28 const char *html_home =
29     "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n"
30     "<html><body>"
31     "<h1>May tinh cua toi</h1>"
32     "</body></html>\r\n"
```



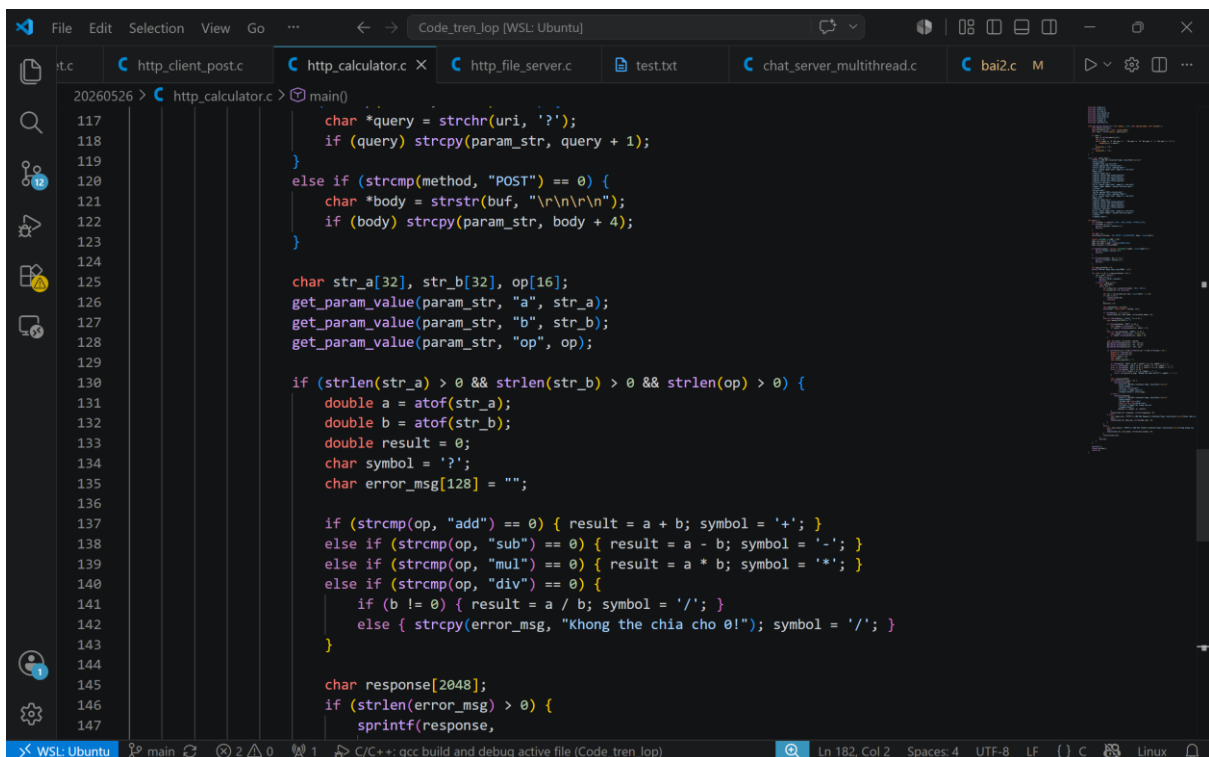
```
20260526 > http_calculator.c > main()
30     "<html><body>"
31     "<h1>May tinh cua toi</h1>"
32     "<b>Thu nghiem GET:</b><br><br>"
33     "<form action='/calc' method='GET'"
34     "So A: <input type='text' name='a'> <br><br>"
35     "Phep tinh: "
36     "<select name='op'"
37     "<option value='add'>Cong</option>"
38     "<option value='sub'>Tru</option>"
39     "<option value='mul'>Nhan</option>"
40     "<option value='div'>Chia</option>"
41     "</select> <br><br>"
42     "So B: <input type='text' name='b'> <br><br>"
43     "<input type='submit' value='Tinh ket qua'"
44     "</form>"
45     "<br><br>"
46     "<b>Thu nghiem POST:</b><br><br>"
47     "<form action='/calc' method='POST'"
48     "So A: <input type='text' name='a'> <br><br>"
49     "Phep tinh: "
50     "<select name='op'"
51     "<option value='add'>Cong</option>"
52     "<option value='sub'>Tru</option>"
53     "<option value='mul'>Nhan</option>"
54     "<option value='div'>Chia</option>"
55     "</select> <br><br>"
56     "So B: <input type='text' name='b'> <br><br>"
57     "<input type='submit' value='Tinh ket qua'"
58     "</form>"
59     "</body></html>";
60
```



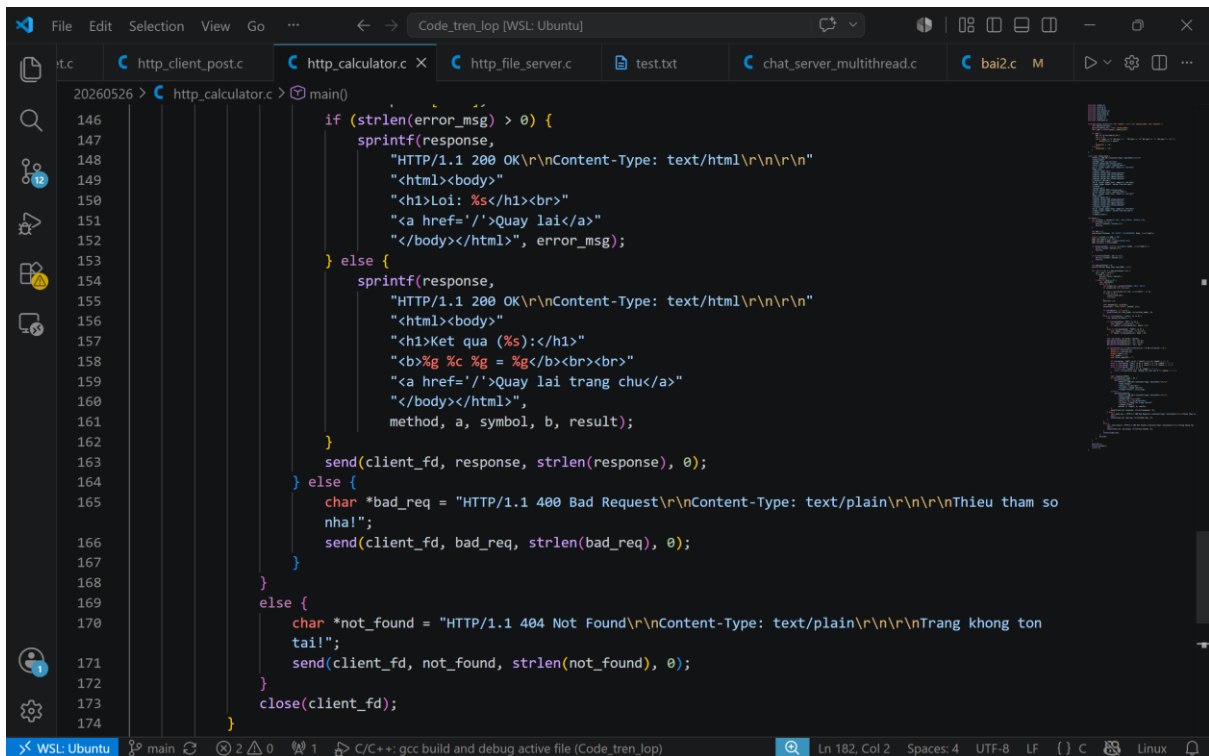
```
20260526 > http_calculator.c > main()
59     "</body></html>";
60
61     int main() {
62         int listener = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
63         if (listener == -1) {
64             perror("socket() failed.\n");
65             exit(1);
66         }
67
68         int opt = 1;
69         setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
70
71         struct sockaddr_in addr = {0};
72         addr.sin_family = AF_INET;
73         addr.sin_addr.s_addr = htonl(INADDR_ANY);
74         addr.sin_port = htons(8080);
75
76         if (bind(listener, (struct sockaddr*)&addr, sizeof(addr))) {
77             perror("bind() failed.\n");
78             exit(1);
79         }
80
81         if (listen(listener, 10) == -1) {
82             perror("listen() failed.\n");
83             exit(1);
84         }
85
86         int num_processes = 8;
87         printf("Server dang chay cong 8080...\n");
88
89         for (int i = 0; i < num_processes; i++) {
```



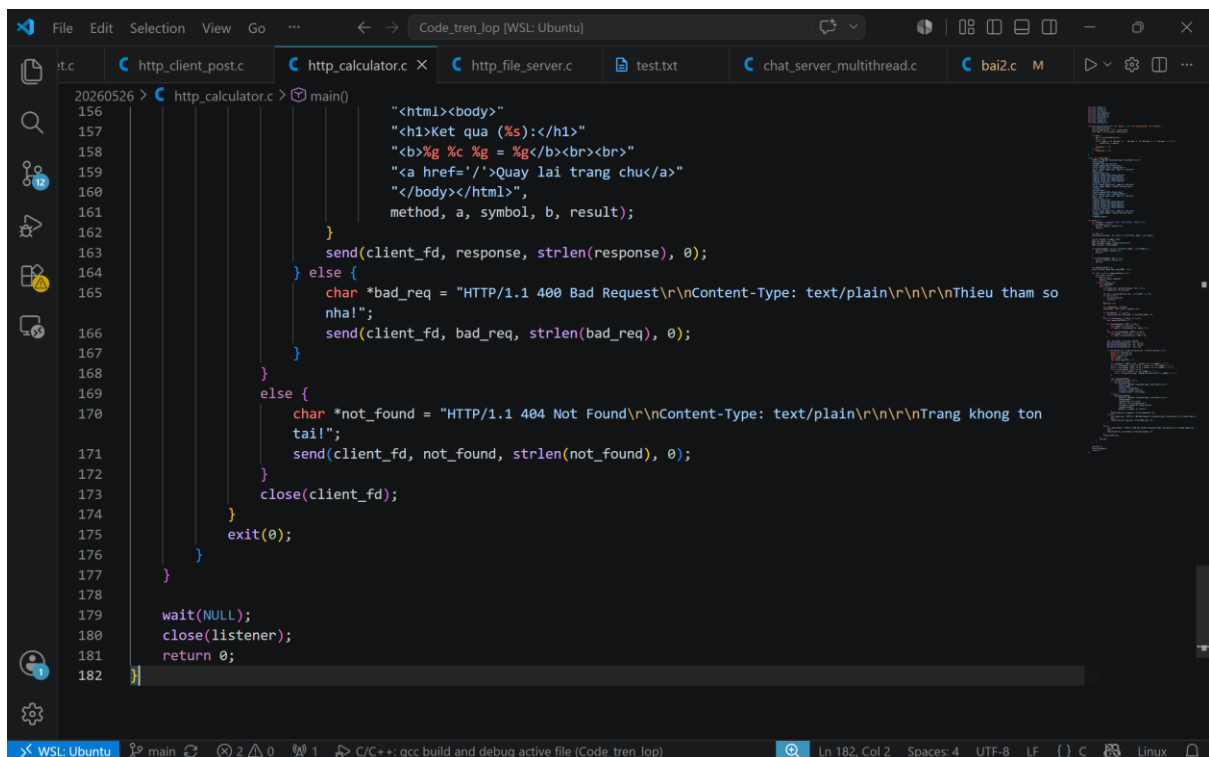
```
20260526 > http_calculator.c > main()
88
89 for (int i = 0; i < num_processes; i++) {
90     pid_t pid = fork();
91     if (pid == -1) {
92         perror("fork() failed");
93         exit(1);
94     } else if (pid == 0) {
95         char buf[2048];
96         while (1) {
97             int client_fd = accept(listener, NULL, NULL);
98             if (client_fd < 0) continue;
99
100             int ret = recv(client_fd, buf, sizeof(buf) - 1, 0);
101             if (ret <= 0) {
102                 close(client_fd);
103                 continue;
104             }
105             buf[ret] = 0;
106
107             char method[16], uri[256];
108             sscanf(buf, "%15s %255s", method, uri);
109
110             if (strcmp(uri, "/") == 0) {
111                 send(client_fd, html_home, strlen(html_home), 0);
112             }
113             else if (strncmp(uri, "/calc", 5) == 0) {
114                 char param_str[1024] = "";
115
116                 if (strcmp(method, "GET") == 0) {
117                     char *query = strchr(uri, '?');
118                     if (query) strcpy(param_str, query + 1);
```



```
117         char *query = strchr(uri, '?');
118         if (query) strcpy(param_str, query + 1);
119     }
120     else if (strcmp(method, "POST") == 0) {
121         char *body = strstr(buf, "\r\n\r\n");
122         if (body) strcpy(param_str, body + 4);
123     }
124
125     char str_a[32], str_b[32], op[16];
126     get_param_value(param_str, "a", str_a);
127     get_param_value(param_str, "b", str_b);
128     get_param_value(param_str, "op", op);
129
130     if (strlen(str_a) > 0 && strlen(str_b) > 0 && strlen(op) > 0) {
131         double a = atof(str_a);
132         double b = atof(str_b);
133         double result = 0;
134         char symbol = '?';
135         char error_msg[128] = "";
136
137         if (strcmp(op, "add") == 0) { result = a + b; symbol = '+'; }
138         else if (strcmp(op, "sub") == 0) { result = a - b; symbol = '-'; }
139         else if (strcmp(op, "mul") == 0) { result = a * b; symbol = '*'; }
140         else if (strcmp(op, "div") == 0) {
141             if (b != 0) { result = a / b; symbol = '/'; }
142             else { strcpy(error_msg, "Khong the chia cho 0!"); symbol = '/'; }
143         }
144
145         char response[2048];
146         if (strlen(error_msg) > 0) {
147             sprintf(response,
```



```
20260526 > http_calculator.c > main()
146     if (strlen(error_msg) > 0) {
147         sprintf(response,
148             "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n"
149             "<html><body>"
150             "<h1>Loi: %s</h1><br>"
151             "<a href='/'>Quay lai</a>"
152             "</body></html>", error_msg);
153     } else {
154         sprintf(response,
155             "HTTP/1.1 200 OK\r\nContent-Type: text/html\r\n\r\n"
156             "<html><body>"
157             "<h1>Ket qua (%s):</h1>"
158             "<b>%g %c %g = %g</b><br><br>"
159             "<a href='/'>Quay lai trang chuc</a>"
160             "</body></html>",
161             method, a, symbol, b, result);
162     }
163     send(client_fd, response, strlen(response), 0);
164 } else {
165     char *bad_req = "HTTP/1.1 400 Bad Request\r\nContent-Type: text/plain\r\n\r\nThieu tham so\nha!";
166     send(client_fd, bad_req, strlen(bad_req), 0);
167 }
168 }
169 else {
170     char *not_found = "HTTP/1.1 404 Not Found\r\nContent-Type: text/plain\r\n\r\nTrang khong ton\n\n";
171     send(client_fd, not_found, strlen(not_found), 0);
172 }
173 close(client_fd);
174 }
```



```
175     }
176 }
177 }
178
179 wait(NULL);
180 close(listener);
181 return 0;
182 }
```

Kết quả:

May tinh cua toi

Thu nhien GET:

So A:

Phep tinh:

So B:

Thu nhien POST:

So A:

Phep tinh:

So B:

Ket qua (GET):

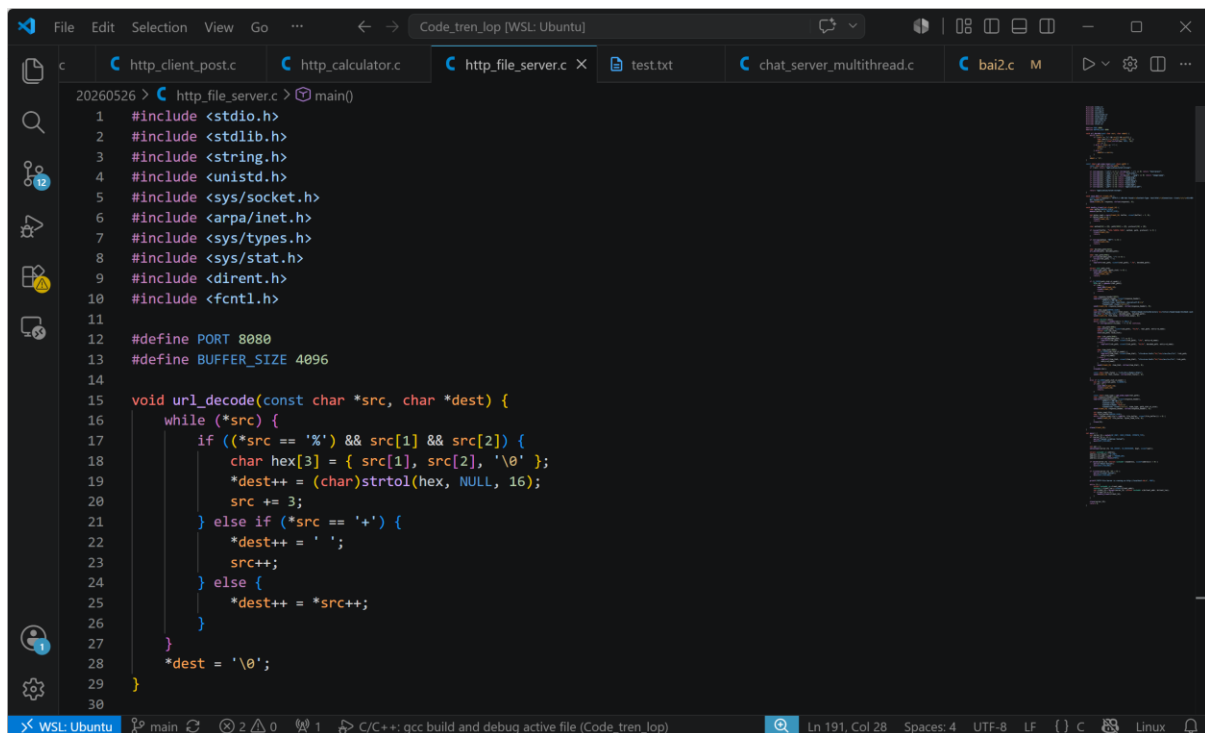
120 + 36 = 156

[Quay lai trang chu](#)

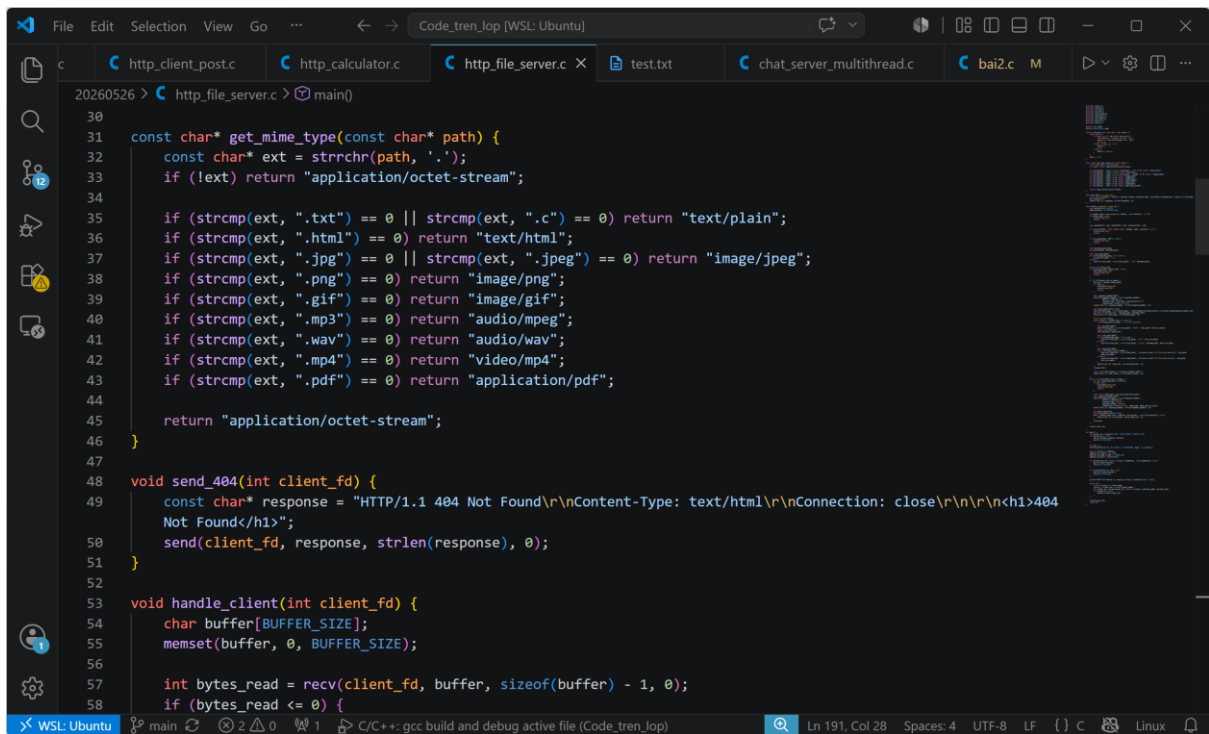
Bài 2: Lập trình ứng dụng máy chủ HTTP file với các chức năng:

- + Khi trình duyệt yêu cầu thư mục gốc, trả về nội dung là trang web chứa danh sách các liên kết đến thư mục và tập tin của thư mục hiện tại trên server. In đậm đối với thư mục và in nghiêng đối với file.
- + Nếu người dùng nhấn vào liên kết thư mục, server trả về nội dung của thư mục con đó (tương tự như trên)
- + Nếu người dùng nhấn vào liên kết file, thì trả về nội dung file. Các file được hỗ trợ bao gồm file văn bản, file audio, file video, file ảnh.

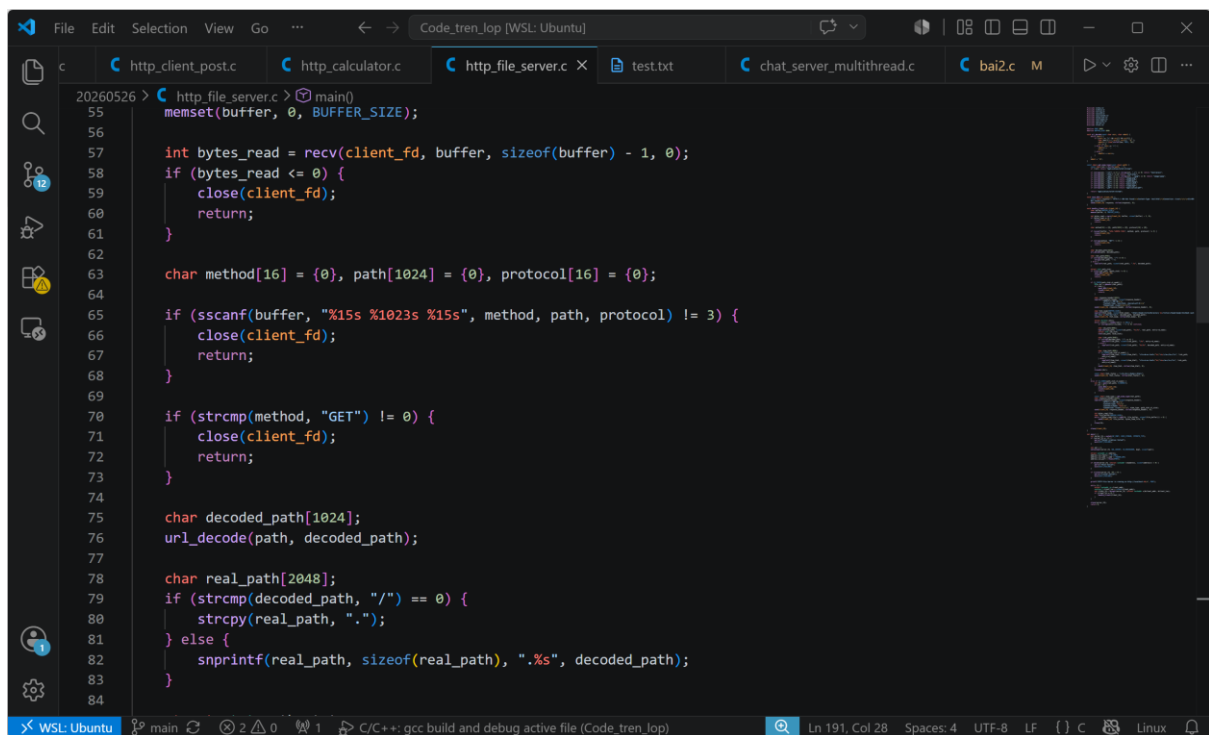
Code:



```
20260526 > C http_file_server.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <sys/socket.h>
6  #include <arpa/inet.h>
7  #include <sys/types.h>
8  #include <sys/stat.h>
9  #include <dirent.h>
10 #include <fcntl.h>
11
12 #define PORT 8080
13 #define BUFFER_SIZE 4096
14
15 void url_decode(const char *src, char *dest) {
16     while (*src) {
17         if ((*src == '%') && src[1] && src[2]) {
18             char hex[3] = { src[1], src[2], '\0' };
19             *dest++ = (char)strtol(hex, NULL, 16);
20             src += 3;
21         } else if (*src == '+') {
22             *dest++ = ' ';
23             src++;
24         } else {
25             *dest++ = *src++;
26         }
27     }
28     *dest = '\0';
29 }
30
```

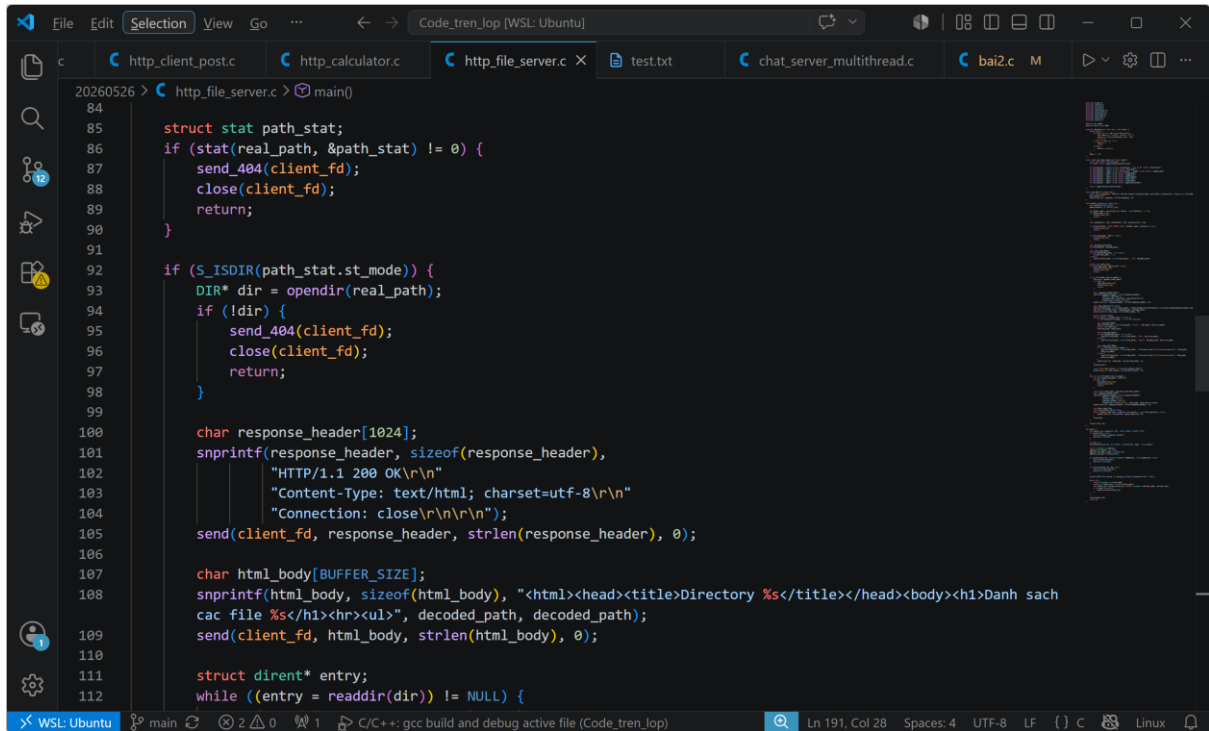


```
20260526 > C http_file_server.c > main()
30
31 const char* get_mime_type(const char* path) {
32     const char* ext = strrchr(path, '.');
33     if (!ext) return "application/octet-stream";
34
35     if (strcmp(ext, ".txt") == 0 || strcmp(ext, ".c") == 0) return "text/plain";
36     if (strcmp(ext, ".html") == 0) return "text/html";
37     if (strcmp(ext, ".jpg") == 0 || strcmp(ext, ".jpeg") == 0) return "image/jpeg";
38     if (strcmp(ext, ".png") == 0) return "image/png";
39     if (strcmp(ext, ".gif") == 0) return "image/gif";
40     if (strcmp(ext, ".mp3") == 0) return "audio/mpeg";
41     if (strcmp(ext, ".wav") == 0) return "audio/wav";
42     if (strcmp(ext, ".mp4") == 0) return "video/mp4";
43     if (strcmp(ext, ".pdf") == 0) return "application/pdf";
44
45     return "application/octet-stream";
46 }
47
48 void send_404(int client_fd) {
49     const char* response = "HTTP/1.1 404 Not Found\r\nContent-Type: text/html\r\nConnection: close\r\n\r\n<h1>404
50     Not Found</h1>";
51     send(client_fd, response, strlen(response), 0);
52 }
53
54 void handle_client(int client_fd) {
55     char buffer[BUFFER_SIZE];
56     memset(buffer, 0, BUFFER_SIZE);
57
58     int bytes_read = recv(client_fd, buffer, sizeof(buffer) - 1, 0);
59     if (bytes_read <= 0) {
```

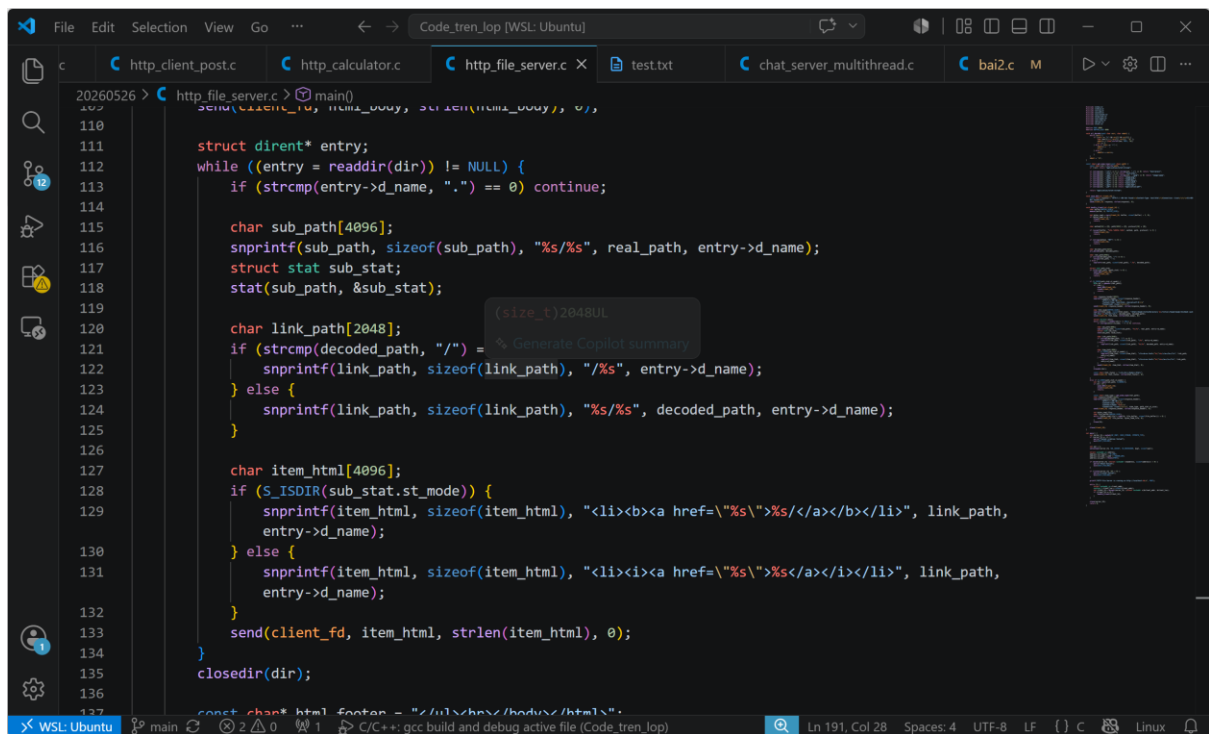


```

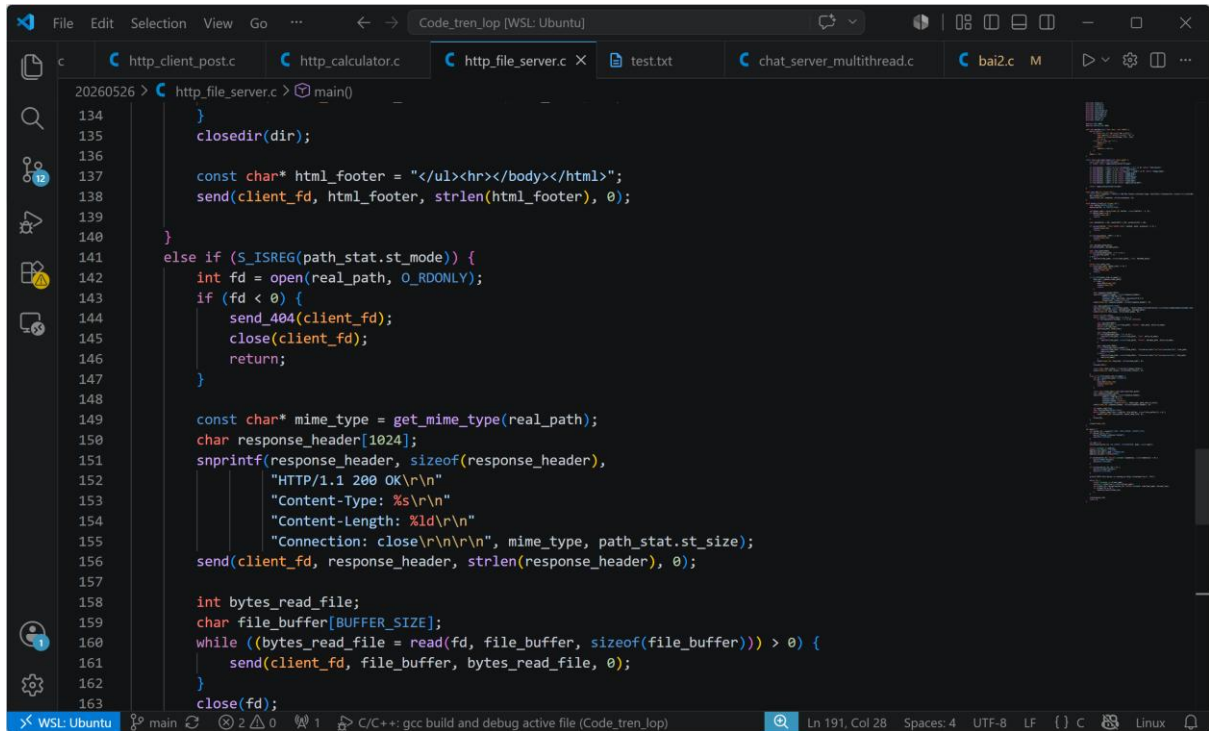
60         close(client_fd);
61         return;
62     }
63
64     char method[16] = {0}, path[1024] = {0}, protocol[16] = {0};
65
66     if (sscanf(buffer, "%15s %1023s %15s", method, path, protocol) != 3) {
67         close(client_fd);
68         return;
69     }
70
71     if (strcmp(method, "GET") != 0) {
72         close(client_fd);
73         return;
74     }
75
76     char decoded_path[1024];
77     url_decode(path, decoded_path);
78
79     char real_path[2048];
80     if (strcmp(decoded_path, "/") == 0) {
81         strcpy(real_path, ".");
82     } else {
83         snprintf(real_path, sizeof(real_path), ".%s", decoded_path);
84     }
85 }
```

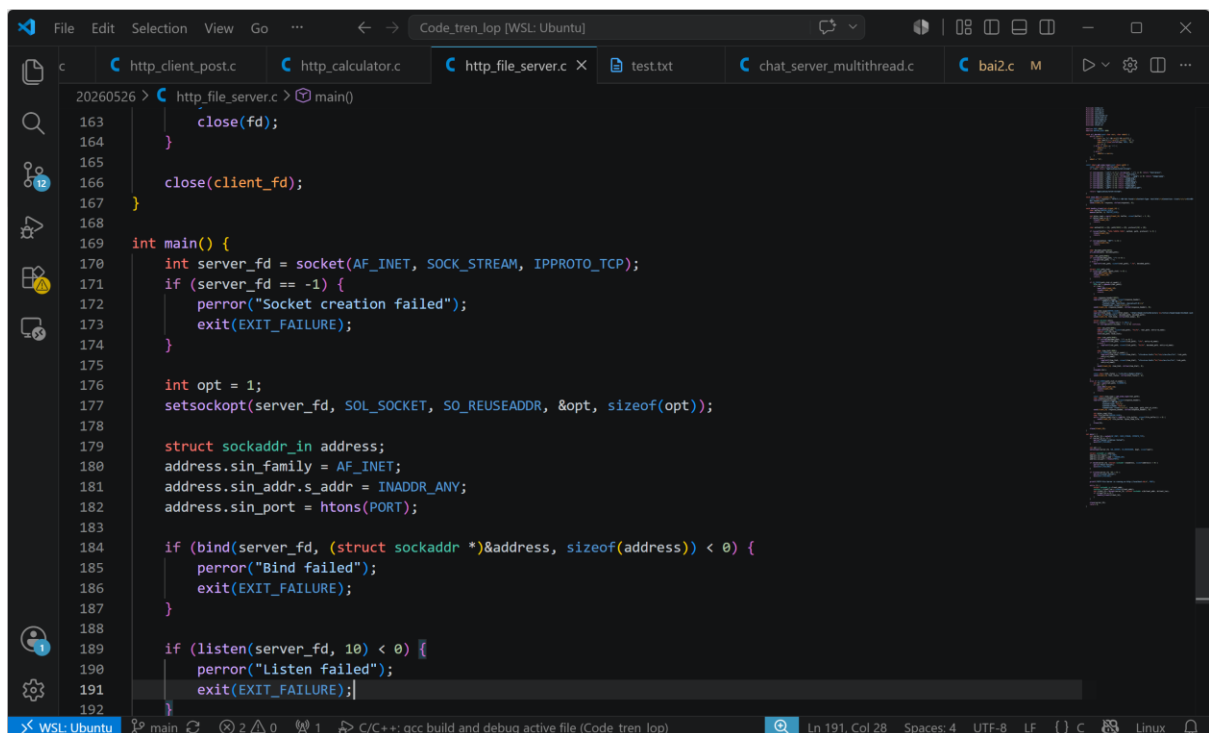
```
20260526 > C http_file_server.c > main()
84
85 struct stat path_stat;
86 if (stat(real_path, &path_stat) != 0) {
87     send_404(client_fd);
88     close(client_fd);
89     return;
90 }
91
92 if (S_ISDIR(path_stat.st_mode)) {
93     DIR* dir = opendir(real_path);
94     if (!dir) {
95         send_404(client_fd);
96         close(client_fd);
97         return;
98     }
99
100     char response_header[1024];
101     snprintf(response_header, sizeof(response_header),
102              "HTTP/1.1 200 OK\r\n"
103              "Content-Type: text/html; charset=utf-8\r\n"
104              "Connection: close\r\n\r\n");
105     send(client_fd, response_header, strlen(response_header), 0);
106
107     char html_body[BUFFER_SIZE];
108     snprintf(html_body, sizeof(html_body), "<html><head><title>Directory %s</title></head><body><h1>Danh sach
cac file %s</h1><hr><ul>", decoded_path, decoded_path);
109     send(client_fd, html_body, strlen(html_body), 0);
110
111     struct dirent* entry;
112     while ((entry = readdir(dir)) != NULL) {
```



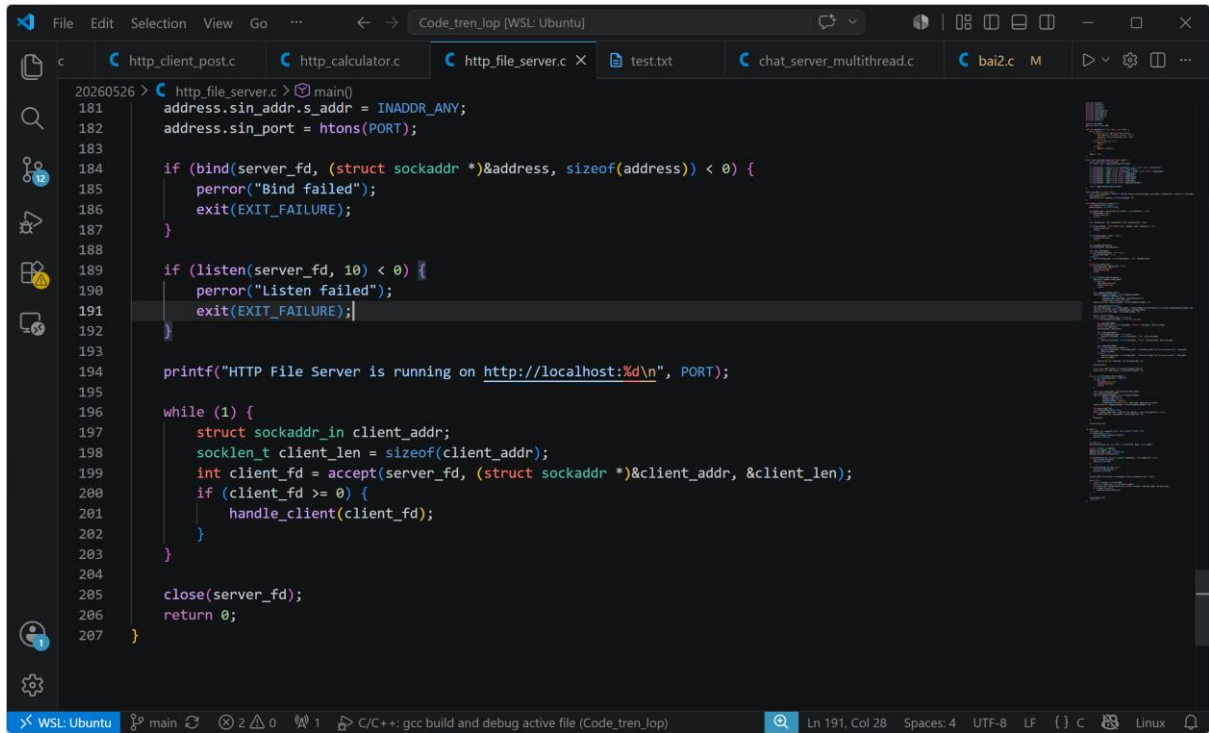
```
110     send(client_fd, html_body, strlen(html_body), 0);
111
112     struct dirent* entry;
113     while ((entry = readdir(dir)) != NULL) {
114         if (strcmp(entry->d_name, ".") == 0) continue;
115
116         char sub_path[4096];
117         snprintf(sub_path, sizeof(sub_path), "%s/%s", real_path, entry->d_name);
118         struct stat sub_stat;
119         stat(sub_path, &sub_stat);
120
121         char link_path[2048];
122         if (strcmp(decoded_path, "/") == 0) {
123             snprintf(link_path, sizeof(link_path), "/%s", entry->d_name);
124         } else {
125             snprintf(link_path, sizeof(link_path), "%s/%s", decoded_path, entry->d_name);
126         }
127
128         char item_html[4096];
129         if (S_ISDIR(sub_stat.st_mode)) {
130             snprintf(item_html, sizeof(item_html), "<li><b><a href=\"%s\">%s</a></b></li>", link_path,
entry->d_name);
131         } else {
132             snprintf(item_html, sizeof(item_html), "<li><i><a href=\"%s\">%s</a></i></li>", link_path,
entry->d_name);
133         }
134         send(client_fd, item_html, strlen(item_html), 0);
135     }
136     closedir(dir);
137
138     const char* html_footer = "</ul><body></html>";
```



```
20260526 > http_file_server.c > main()
134     }
135     closedir(dir);
136
137     const char* html_footer = "</ul><hr></body></html>";
138     send(client_fd, html_footer, strlen(html_footer), 0);
139
140 }
141
142 else if (S_ISREG(path_stat.st_mode)) {
143     int fd = open(real_path, O_RDONLY);
144     if (fd < 0) {
145         send_404(client_fd);
146         close(client_fd);
147         return;
148     }
149
150     const char* mime_type = get_mime_type(real_path);
151     char response_header[1024];
152     snprintf(response_header, sizeof(response_header),
153              "HTTP/1.1 200 OK\r\n"
154              "Content-Type: %s\r\n"
155              "Content-Length: %ld\r\n"
156              "Connection: close\r\n\r\n", mime_type, path_stat.st_size);
157     send(client_fd, response_header, strlen(response_header), 0);
158
159     int bytes_read_file;
160     char file_buffer[BUFFER_SIZE];
161     while ((bytes_read_file = read(fd, file_buffer, sizeof(file_buffer))) > 0) {
162         send(client_fd, file_buffer, bytes_read_file, 0);
163     }
164     close(fd);
165 }
```



```
163     close(fd);
164 }
165
166 close(client_fd);
167 }
168
169 int main() {
170     int server_fd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
171     if (server_fd == -1) {
172         perror("Socket creation failed");
173         exit(EXIT_FAILURE);
174     }
175
176     int opt = 1;
177     setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt, sizeof(opt));
178
179     struct sockaddr_in address;
180     address.sin_family = AF_INET;
181     address.sin_addr.s_addr = INADDR_ANY;
182     address.sin_port = htons(PORT);
183
184     if (bind(server_fd, (struct sockaddr*)&address, sizeof(address)) < 0) {
185         perror("Bind failed");
186         exit(EXIT_FAILURE);
187     }
188
189     if (listen(server_fd, 10) < 0) {
190         perror("Listen failed");
191         exit(EXIT_FAILURE);
192     }
193 }
```



```
20260526 > http_file_server.c > main()
181 address.sin_addr.s_addr = INADDR_ANY;
182 address.sin_port = htons(PORT);
183
184 if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
185     perror("Bind failed");
186     exit(EXIT_FAILURE);
187 }
188
189 if (listen(server_fd, 10) < 0) {
190     perror("Listen failed");
191     exit(EXIT_FAILURE);
192 }
193
194 printf("HTTP File Server is running on http://localhost:%d\n", PORT);
195
196 while (1) {
197     struct sockaddr_in client_addr;
198     socklen_t client_len = sizeof(client_addr);
199     int client_fd = accept(server_fd, (struct sockaddr *)&client_addr, &client_len);
200     if (client_fd >= 0) {
201         handle_client(client_fd);
202     }
203 }
204
205 close(server_fd);
206 return 0;
207 }
```

Kết quả:

